Scalable Network I/O: From select to io_uring

CS 331 Computer Networks · Team T008 · Project ID 13

Chitiveli Hemcharan Varma	Gella Naga Sai Krishna	Guransh Kaur Saran	Kowel P Laloo	Pallav Biyala
Roll No. 2411095	Roll No. 24110122	Roll No. 24110127	Roll No. 24110168	Roll No. 24110234

I. Introduction

Network servers must handle thousands of concurrent connections with few threads. Linux offers four generations of event-notification interfaces for this: select(2) and poll(2), which scan every watched descriptor on each call; epoll(7), which reports only ready descriptors; and io_uring, which replaces readiness notification with submission and completion queues shared with the kernel [1]–[4]. Which one to use is a recurring engineering decision, and the answer depends on the workload.

We build two servers – an echo server and an HTTP/1.1 file transfer server – on one shared core and one request-based I/O interface, with exactly one of the four backends linked into each binary. Because the application code is identical across backends, performance differences can be attributed to the event-notification layer. Both servers are benchmarked with an automated, reproducible runner.

Main contributions: (1) a backend-independent I/O contract (io.h) with four textbook backends; (2) an echo server and an HTTP file server built on it; (3) an experiment runner that pins CPUs, validates every run and archives raw evidence; (4) 360 file-transfer runs across five experiments, with an analysis of when and why the backends differ.

[ECHO SERVER – TO FILL] Add 1–2 sentences on the echo-server experiments and their headline result (e.g. which backend scales best with message size or connection count).

II. Problem Statement

Problem. For a single-threaded, event-driven server on Linux, how do select, poll, epoll and io_uring compare in throughput, latency and CPU cost, and how does the answer change with concurrency, payload size, connection reuse, buffer settings, slow clients and TCP options?

Inputs: backend; number of client connections (1–1000); file size (4 KiB–100 MiB) or echo message size; keep-alive vs Connection: close; chunk and socket-buffer sizes; number of rate-limited slow readers; TCP_NODELAY. **Outputs:** throughput (responses/s, MB/s), latency (mean, p99), server CPU per response and server-core utilisation.

Assumptions and scope: one server thread pinned to one core; client and server on one machine over loopback TCP; files served from the page cache; HTTP/1.1 GET only, no TLS. Each backend uses its mechanism in the standard way, with optional features behind flags. Multi-threaded servers, zero-copy sending, real NICs/WANs and TLS are out of scope.

Gap. Many comparisons change the application together with the mechanism, or use micro-benchmarks. We hold the application constant, isolate the event layer, and report CPU cost and server saturation, not only throughput.

[ECHO SERVER – TO FILL] State the echo-server problem precisely: request/response format, message sizes, what is measured (round trips/s, latency), and any echo-specific assumptions.

III. Project Objectives

- O1. Implement four backends behind one interface; both servers pass the correctness suite on all four backends (file server: 9/9 checks per backend, 36/36 in total).
- O2. Build a reproducible runner: 5 repetitions per configuration, 10 s warm-up, 30 s measurement, automatic acceptance checks, raw logs archived.
- O3. Measure file-transfer throughput and latency at 1, 100 and 1000 connections and for 4 KiB, 64 KiB and 1 MiB files (E5a).
- O4. Quantify the effect of connection reuse (E5b), chunk and socket-buffer sizes (E7), slow readers (E8) and TCP_NODELAY (E9).
- O5. Explain the results with server CPU time, core utilisation and syscall counters, not only throughput.

[ECHO SERVER – TO FILL] Add the echo-server objectives (measurable, e.g. message sizes and concurrency levels tested, number of runs).

IV. Architecture and Design

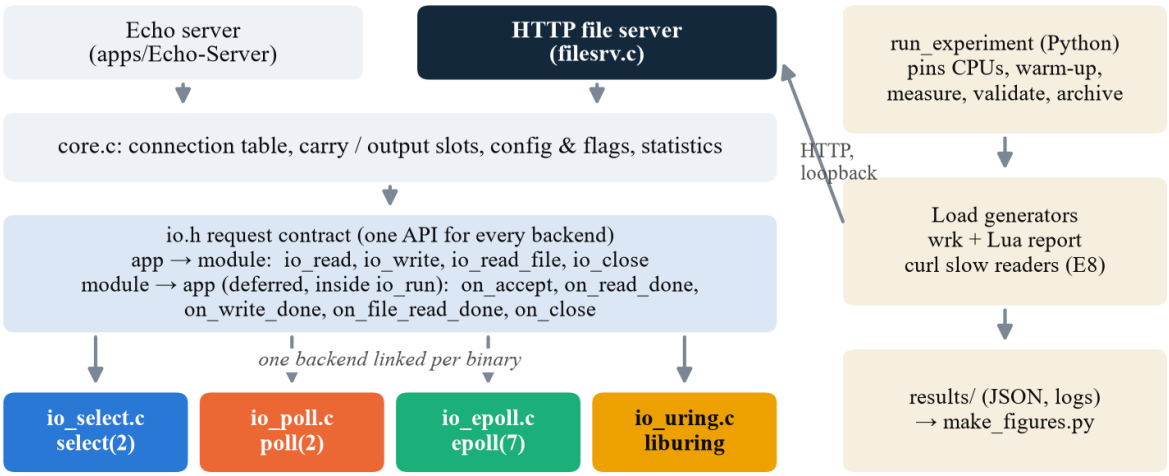


Fig. 1. System architecture. Applications talk only to io.h; one backend is linked into each binary. The runner (right) drives load over loopback and archives results.

Layers. Applications implement callbacks and never include epoll.h, poll.h or liburing.h. *core.c* holds the connection table, a per-connection carry slot (bytes of the next request), output slots for unsent data, flag validation and statistics counters (dumped on SIGUSR1). *io.h* is a request-based contract: the app asks for io_read, io_write, io_read_file or io_close, and the module reports completion through callbacks. Readiness backends wait for readiness and then call recv/send; io_uring queues the operation. A request that finishes immediately is queued and delivered at the top of the next loop iteration, which prevents callback recursion and behaves the same in all four modules.

Backends. select rebuilds its fd_sets every iteration and is limited to FD_SETSIZE (1024) descriptors; poll keeps a compact pollfd array; epoll is level-triggered and calls epoll_ctl only when a send is incomplete; io_uring keeps one ACCEPT outstanding, issues a RECV or SEND per connection, and tags user_data with a generation number to reject stale completions.

File server. Each connection runs a small state machine: receive and parse the request (GET, safe file name, Connection header) → send the header → repeat {read a file chunk of up to 64 KiB → write it} → finish, then either wait for the next request (keep-alive) or close. It returns 200, 400, 404 or 405. The header and the file data never share a buffer, because buffer ownership differs between backends.

Table I. Key design decisions and trade-offs.

Decision	Why	Trade-off
Request-based API (not readiness)	Fits io_uring natively; same app code on all backends	Readiness backends must emulate completions (extra staging copy)
One backend per binary (compile time)	No dispatch overhead; clean separation	Eight binaries to build and test
Single thread, pinned core	Isolates the mechanism; repeatable	Limits load; a multi-core server would scale further
read() + write(), no sendfile	Backend performs every I/O operation	Copying dominates large-file cost
Sequential keep-alive, no pipelining	Simple, correct framing	Less realistic than production HTTP servers

[ECHO SERVER – TO FILL] Describe the echo server design: message framing (--msg-size), how it uses io_read / io_write and the carry slot, and how a full echo round trip maps onto the callbacks.

V. Implementation

Stack. C17 compiled with GCC (GNU Make builds 8 binaries: 2 apps × 4 backends); liburing for io_uring; Python 3 for the experiment runner; wrk with a Lua script for latency percentiles; curl for rate-limited slow readers; matplotlib for analysis. The file server is about 800 lines of C. Code: https://github.com/Pallav-Biyala/CS331-T008-NetworkIO.

Core of the file-server response path (callbacks are always invoked from io_run):

```
on_read_done(c, buf, n):  append buf to carry; if a full request is present:
    parse request -> status, file;  hdr = io get buf(c);  build header in hdr
    io write(c, hdr)                          # header first, separate buffer
on write done(c):
    if bytes_left > 0:  b = io_get_buf(c);  io_read_file(c, b, min(chunk, left), fd, off)
    elif keep_alive:    io_read(c, buf_size)  # wait for next request
    else:               io_close(c)
on_file_read_done(c, b, n):  off += n;  left -= n;  io_write(c, b, n)
```

Challenges and solutions. (i) Buffer lifetime differs (shared scratch buffer on readiness backends, per-connection buffer on io_uring), so request bytes are copied into the carry slot before the header is built, and a file chunk is read only after the header write completes. (ii) Callback recursion (on_read_done → io_write → on_write_done → …) is avoided with a deferred-completion queue and a zero wait timeout. (iii) Completions arriving after a close are rejected using generation-tagged io_uring user_data; close waits for the in-flight operation. (iv) The first E5b Connection: close campaign was rejected, so all four backends were rerun with --max-conns 300; E8 needed a 90 s window so that 100 MiB downloads could finish.

[ECHO SERVER – TO FILL] Add echo-server implementation notes and challenges (e.g. partial reads/writes, framing large messages, any backend-specific bug you fixed).

VI. Testbed Setup

Table II. Experimental environment.

Item	Value
Hardware	Intel Core i5-10300H (4 cores / 8 threads, 2.5 GHz), ~8 GB RAM available to WSL2
OS / software	Windows 11 + WSL2, Linux 6.18.54-microsoft-standard-WSL2, glibc 2.43, Python 3.14.4, GCC (C17), liburing, wrk
Topology	Single host, loopback 127.0.0.1. Server pinned to CPU 1; wrk (4 threads) pinned to CPUs 2–5
Kernel limits	net.core.somaxconn = 4096; open files 10,240; kernel.io_uring_disabled = 0
Workloads	Deterministic fixtures of 4 KiB, 64 KiB, 1 MiB, 4 MiB and 100 MiB (make bench-fixtures)
Timing	10 s warm-up, 30 s measured (90 s in E8), 10 s cool-down; 5 repetitions; configuration order randomised

Table III. File-transfer experiments (all on all four backends).

Exp.	Varied	Fixed	Runs
E5a	Connections 1/100/1000 (4 KiB); 64 KiB at 10; 1 MiB at 100	Keep-alive, 64 KiB chunks	100
E5b	Connection: close vs keep-alive	4 KiB, 100 conns, --max-conns 300	40
E7	Chunk 16 KiB; SO_RCVBUF 512 KiB; SO_SNDBUF 64 KiB	4 MiB, 100 conns, app buffer 256 KiB	80
E8	0 / 10 / 100 slow readers (100 KiB/s)	100 MiB, 100 fast wrk connections	60
E9	TCP_NODELAY on / off	4 KiB and 64 KiB, 100 conns	80

[ECHO SERVER – TO FILL] Add the echo-server workloads and parameters (message sizes, connection counts, client tool and command, number of runs) as rows or a separate table.

VII. Evaluation

Method. Each point is the median of 5 accepted runs; whiskers show the inter-quartile range. 357 of 360 file-transfer runs were accepted. Throughput is measured by wrk; latency is wrk's mean time to the last byte. Because wrk is closed-loop (each connection sends its next request as soon as the previous one completes), mean latency ≈ connections ÷ throughput (Little's law [5]). Server CPU and core utilisation come from /proc.

A. Echo server

[ECHO SERVER – TO FILL] Present the echo-server results here: one figure with 2–3 panels (e.g. throughput vs connections, latency vs message size) plus 150–250 words of discussion. Suggested budget: about 1 page including the figure. Use the same colours as the file-transfer figures (select blue, poll orange, epoll green, io_uring yellow) and the same median + IQR convention.

B. File transfer: concurrency and file size (E5a)

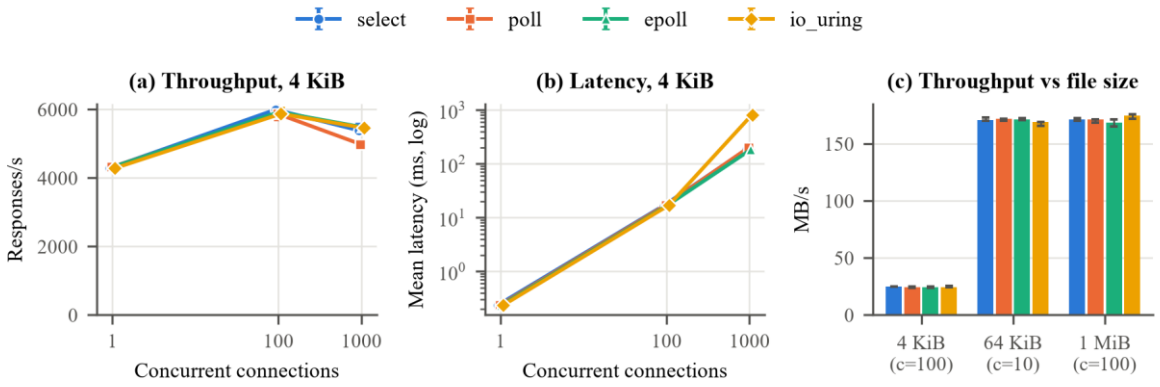


Fig. 2. E5a. (a) Throughput and (b) mean latency vs concurrency for a 4 KiB file; (c) throughput vs file size. Medians of 5 runs; whiskers show the IQR.

From 1 to 100 connections throughput rises 37% (4.3k → 5.9k req/s) on every backend; at 1000 connections it falls 7–15%, and poll falls most (4,989 vs 5,484 req/s for epoll) because it rebuilds and rescans its 1000-entry pollfd array three times per loop in user space. Latency follows Little's law (100/5,925 ≈ 17 ms; 1000/5,484 ≈ 182 ms). The exception is io_uring at 1000 connections: the same throughput as epoll, but a mean of 818 ms and a p99 of 13.5 s in all 5 runs – some connections are starved for seconds, a fairness problem. Small files are request-bound (~24.5 MB/s at 4 KiB); 64 KiB and 1 MiB files reach the same ~170 MB/s ceiling on all backends. Note that the 64 KiB point used 10 connections.

C. Connection reuse and buffer sizes (E5b, E7)

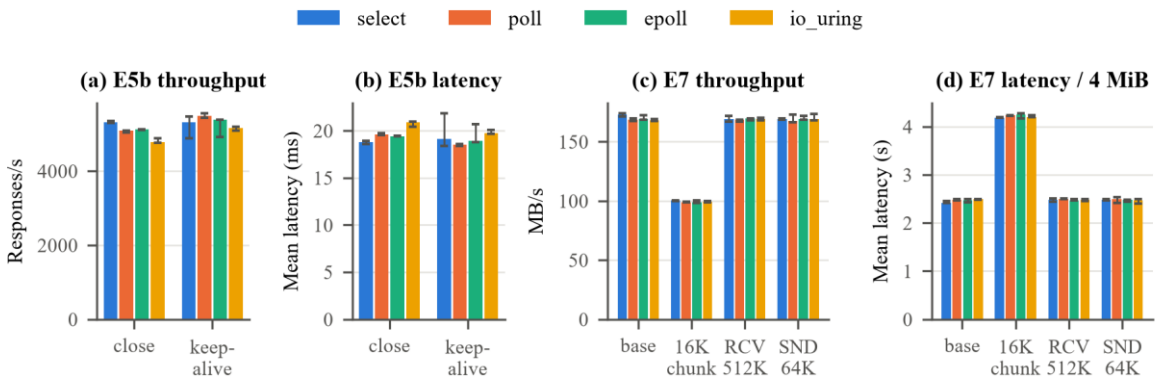


Fig. 3. (a, b) E5b: Connection: close vs keep-alive, 4 KiB, 100 connections. (c, d) E7: buffer settings, 4 MiB file, 100 connections.

E5b. Keep-alive measured +5 to +8% for poll, epoll and io_uring and 0% for select. io_uring gains most, matching its 15% higher CPU per response when every response opens a new connection. However, the keep-alive half ran 7–12% below the identical E5a configuration in a separate session, so variation between sessions is larger than the effect and the throughput result is inconclusive. On loopback a handshake is almost free; on a real network each new connection adds at least one RTT.

E7. Only the chunk size matters: 16 KiB chunks cut throughput 41% (169 → 99 MB/s) and raise latency per 4 MiB file from 2.5 s to 4.2 s, with 4× the read/write cycles and 2.4× the server CPU per file. The server core is still only ~34% busy, so the loss most likely comes from the client and loopback handling 4× as many smaller writes. SO_RCVBUF has no effect (the server only receives a short GET) and a 64 KiB SO_SNDBUF still holds a whole chunk.

D. Slow readers and TCP_NODELAY (E8, E9)

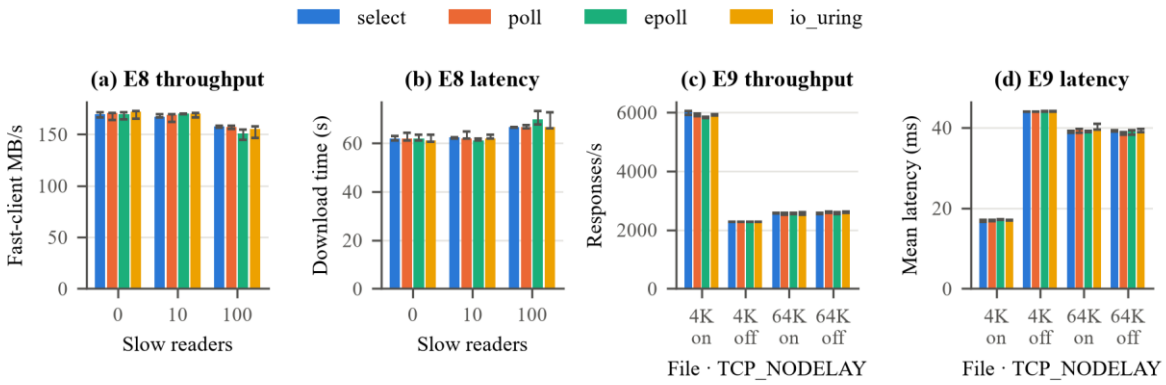


Fig. 4. (a, b) E8: 100 fast clients downloading 100 MiB with 0/10/100 slow readers (100 KiB/s each); at 100 slow readers select, poll and epoll have n = 4. (c, d) E9: TCP_NODELAY on/off, 100 connections.

E8. Ten slow readers have no measurable effect. With 100, fast-client throughput falls 7–11% and each 100 MiB download takes 67–70 s instead of ~62 s. All four servers are non-blocking: a slow reader's write simply waits (EAGAIN or a pending SQE) while the loop serves others, and 100 × 100 KiB/s ≈ 10 MB/s is close to what the fast clients lose. Run 3 at 100 slow readers was rejected for select, poll and epoll (no download finished within 90 s), so these medians are optimistic.

E9. Disabling TCP_NODELAY cuts 4 KiB throughput by 62% (5.9k → 2.27k req/s) and raises latency 2.6× on every backend. The server sends the header and body as two writes; with Nagle's algorithm [6] the small body waits for the header's ACK, which the client delays by ~40 ms [7]. Check: 100 connections ÷ 44 ms ≈ 2,270 req/s, as measured. At 64 KiB only a 53-byte tail is held and the preceding full segment triggers an immediate ACK, so there is no effect.

E. Why the backends look alike, and where they differ

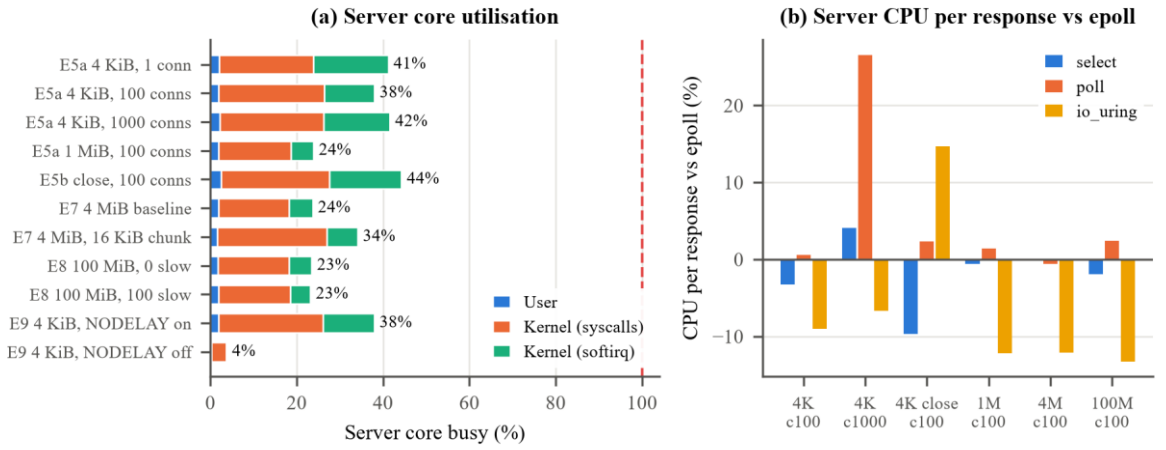


Fig. 5. (a) Busy share of the server's CPU core during the measured window (median of all backends). (b) Server CPU time per response relative to epoll (0 = same as epoll; lower is cheaper).

Fig. 5(a) explains the near-identical throughput: the server core is 4–48% busy in every test, so the limit lies outside the server – most likely wrk, which had only two physical cores, plus the WSL2 loopback path. A backend that saves CPU just leaves the server idle longer. In addition, the backend-specific wait step is tiny: at 100 connections the server waits ~180 times/s and handles ~33 responses per wakeup, against ~17,700 read/write calls/s that are identical code on every backend (Table IV).

Table IV. Wait calls vs data syscalls (server counters, medians).

Configuration	Wait calls/s	Responses per wakeup	read + write calls/s
4 KiB, 100 conns	~180	33	~17,700
4 KiB, 1000 conns	18–55	97–282	~16,000
4 MiB, 100 conns	29–54	~1	~2,660

The backends do differ in CPU cost (Fig. 5(b)): poll needs 27% more CPU per response at 1000 connections; io_uring needs 15% more under Connection: close (per-connection setup) and 12–13% less on large files. Most of that last saving is an implementation effect: our readiness backends pread each chunk into a staging slot and then memcpy it into the send buffer, while io_uring reads straight into the send buffer. On a saturated server these CPU differences would turn into throughput differences.

[ECHO SERVER – TO FILL] Optional (≈ 0.25 page): compare echo vs file transfer – e.g. whether backend differences are larger for the echo server, where per-message work is small and the wait step is a bigger share of the work.

VIII. Summary

We compared select, poll, epoll and io_uring under identical application code for an HTTP file server. In throughput and mean latency the four backends are nearly indistinguishable, because the server was never saturated and the backend-specific step is about 1% of its work; copying data through TCP dominates. The backends do differ in CPU cost (poll +27% at 1000 connections; io_uring −12% on large files, mostly from a copy our readiness backends make) and in tail latency (io_uring p99 13.5 s at 1000 connections). TCP and buffer settings matter far more than the backend: TCP_NODELAY off costs 62% of small-file throughput, and 16 KiB chunks cost 41%.

[ECHO SERVER – TO FILL] Add 2–3 sentences summarising the echo-server findings and how they compare with the file-transfer findings.

Limitations and implementation flaws. (i) The server was never saturated and client CPU was not recorded; wrk ran on two physical cores. (ii) Readiness backends make an extra user-space copy of every file chunk, and io_uring uses only basic features (synchronous pread, no multishot, provided buffers or SQPOLL), so io_uring is not shown at its best. (iii) The io_uring starvation tail at 1000 connections is unexplained. (iv) Binaries were built without an optimisation flag (-O0). (v) Closed-loop load only; WSL2 loopback (~64 KiB segments, no real RTT); E5b halves ran in separate sessions; E5a is not a full grid and E7 never varied the app buffer; E8's window was short; 5 runs and no significance tests.

Future work. Cap the server with a cgroup CPU quota so it becomes the bottleneck and record client CPU; build with -O2; remove the staging copy and add sendfile/splice and a single writev for header and body; use io_uring's advanced features and fix its fairness problem; add a fixed-rate (open-loop) load test; rerun E5b in one session; and repeat key experiments over a real network with more repetitions and confidence intervals.

IX. Reproducibility

Repository: https://github.com/Pallav-Biyala/CS331-T008-NetworkIO (branch project2). Requirements: Linux or WSL2 with GCC, GNU Make, pkg-config, liburing-dev, Python 3, wrk and curl. From the repository root:

```
sudo apt install build-essential pkg-config liburing-dev python3 curl wrk
make && make test                    # build 8 binaries, run correctness suite
make bench-fixtures                  # deterministic test files
python3 code/tools/File-Transfer-Server/run_experiment \
    code/tools/File-Transfer-Server/configs/filesrv-e5a-size-concurrency.json
# also: filesrv-e5b-*-c300.json, filesrv-e7-*.json, filesrv-e8-*.json, filesrv-e9-*.json
python3 report/file-transfer/make_figures.py         # figures + data.json
python3 report/file-transfer/make_report_figures.py  # figures used in this report
```

Raw results of every run (accepted and rejected) are archived under code/results/File Transfer Server/. The detailed analysis is in report/file-transfer/README.md.

[ECHO SERVER – TO FILL] Add the commands and config files used to run and plot the echo-server experiments.

References

[1] J. Axboe, “Efficient IO with io_uring,” 2019. [Online]. Available: https://kernel.dk/io_uring.pdf

[2] Linux man-pages project, “epoll(7), select(2), poll(2),” The Linux Programming Interface manual pages.

[3] G. Banga, J. C. Mogul, and P. Druschel, “A scalable and explicit event delivery mechanism for UNIX,” in Proc. USENIX Annu. Tech. Conf., Monterey, CA, USA, 1999, pp. 253–265.

[4] D. Kegel, “The C10K problem.” [Online]. Available: http://www.kegel.com/c10k.html

[5] J. D. C. Little, “A proof for the queuing formula: L = λW,” Operations Research, vol. 9, no. 3, pp. 383–387, 1961.

[6] J. Nagle, “Congestion control in IP/TCP internetworks,” RFC 896, Jan. 1984.

[7] R. Braden, Ed., “Requirements for Internet hosts – communication layers,” RFC 1122, Oct. 1989.

[8] R. Fielding, M. Nottingham, and J. Reschke, Eds., “HTTP/1.1,” RFC 9112, Jun. 2022.

[9] W. Glozer, “wrk – a HTTP benchmarking tool.” [Online]. Available: https://github.com/wg/wrk

[ECHO SERVER – TO FILL] Add any references used for the echo-server part.

Appendix A: Team Work Distribution and Member Rating

A.1 Work Distribution

Ratings are agreed by the whole team and must be unique (no ties), on a scale of 1–10 (10 = highest contribution).

#	Member Name	Roll No.	Tasks Done	Rating (1–10, unique)
1	Chitiveli Hemcharan Varma	2411095	Benchmarking (from README) – [fill in]	[ ]
2	Gella Naga Sai Krishna	24110122	io_uring backend; file-transfer server evaluation, analysis, figures and report – [confirm]	[ ]
3	Guransh Kaur Saran	24110127	select backend – [fill in]	[ ]
4	Kowel P Laloo	24110168	poll backend – [fill in]	[ ]
5	Pallav Biyala	24110234	epoll backend – [fill in]	[ ]

Appendix B: AI Usage Details

B.1 Prompts

The most significant AI uses are listed below. Prompts and responses are shared in the repository's AI-Logs folder.

#	AI Tool	Purpose
1	Claude (Claude Code, Claude Opus 5.5)	Analysing the file-transfer benchmark data, producing figures, and drafting the file-transfer report and slides; all numbers were checked against the raw run logs
2	[fill in]	[fill in – e.g. echo-server or backend development uses]
3	[fill in]	[fill in]
4	[fill in]	[fill in]